

Shared Memory

Part 2

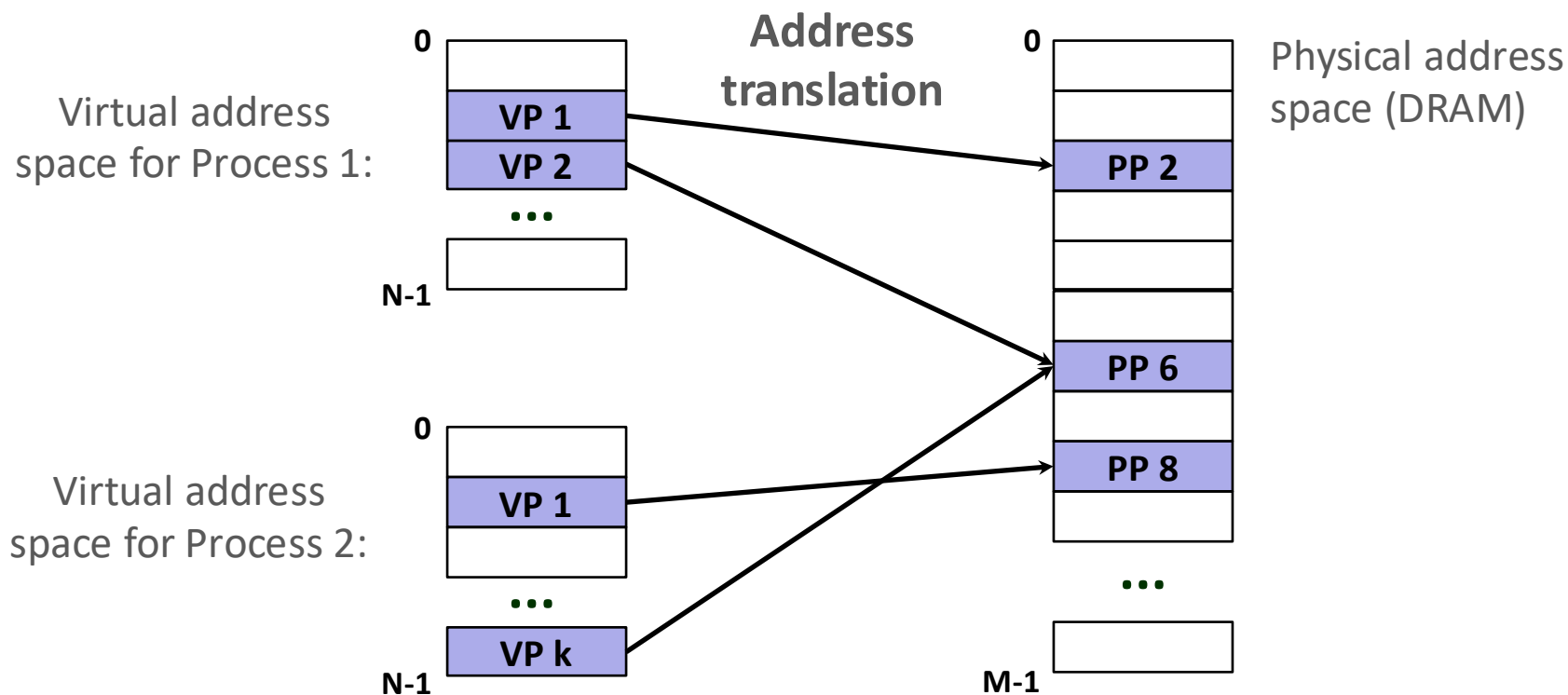
Sistemas de Computadores
Departamento de Engenharia Informática
Instituto Superior de Engenharia do Porto

Luís Nogueira (lmn@isep.ipp.pt)

Recall from last class

■ Memory region which is **shared by several processes**

- Processes with or without a hierarchical link can use this region to communicate



POSIX shared memory

1. Create/open (previous class)

- `shm_open()`, `ftruncate()`, `mmap()`

2. Use the shared memory area (previous class)

- With a pointer, similarly to any dynamic memory block

3. Remove (this class)

- `munmap()`, `close()`, `shm_unlink()`

Create/open a shared memory area

- As we discussed in the previous class, these are the typical steps involved in creating or opening a shared memory area
- 1. Use the `shm_open()` function
 - Creates and opens (or if already exists, just opens) a shared memory area
 - Returns a file descriptor to be used in subsequent functions
- 2. Use the `ftruncate()` function
 - Defines the size of the shared memory area
- 3. Use the `mmap()` function
 - Maps the shared area into the process' address space

Using the shared memory area

- **Shared memory is typically faster and more flexible than message-passing mechanisms (such as pipes or message queues) for applications that exchange large amounts of data**
 - Message-passing requires system calls for each data transfer
 - This introduces kernel overhead and reduces throughput, especially under high-frequency communication

- **However, shared memory has an important drawback**
 - Processes must explicitly synchronize access to avoid race conditions and data corruption
 - Synchronization overhead may dominate if poorly designed
 - This increases design complexity and shifts responsibility to the programmer

Detach and remove

1. The `munmap ()` function

- Detaches the shared memory region from the process's address space (opposite of `mmap ()`)
- Use it after the process no longer needs access to the shared memory

2. The `close ()` function

- Closes the file descriptor obtained via `shm_open ()`
- This does NOT remove the shared memory, only the file descriptor is closed

3. The `shm_unlink ()` function

- Removes the shared memory object from the file system namespace
- Marks the memory for deletion; Deletion occurs only **after all processes have closed** the shared memory

The munmap function

```
#include <sys/mman.h>

int munmap(void *addr, size_t length);
```

- **Disconnects the shared memory area from the process address space**
- **Returns**
 - 0, if successful
 - -1, in case of error

The munmap function

```
#include <sys/mman.h>

int munmap(void *addr, size_t length);
```

- **The `addr` parameter is the pointer returned by `mmap ( )`**
 - Indicating the starting address of the mapped memory region
- **`length` must correspond to a valid mapped region (typically the same size originally mapped)**
 - It does not require being “equal” to the original mapping, but incorrect alignment or partial unmapping can easily introduce bugs

The `close` function

```
#include <unistd.h>

int close(int fd);
```

- Closes the file descriptor associated with an open shared memory object
- `fd` is the file descriptor returned by `shm_open()`
- Returns
 - 0, if successful
 - -1, in case of error

The `shm_unlink` function

```
#include <sys/mman.h>

int shm_unlink(const char *name);
```

- **Removes the shared memory object from the file system**
 - Not immediate, if any process is still using the shared memory
 - Final deletion occurs only after all processes have closed their file descriptors and unmapped the shared memory
- **`name` is the shared memory name used in `shm_open()`**
- **Returns**
 - 0, if successful
 - -1, in case of error

Example – close a shared memory area

- A process should close its shared memory descriptor as soon as it no longer needs it
 - Other processes can still open and access the shared memory independently

```
int fd;
void *addr;                                /* pointer to shared memory */

fd = shm_open("/shmtest", ...);
ftruncate (fd, 100);
addr = mmap(NULL, 100, ...);

...                                         /* use – read/write */

munmap(addr, 100);                         /* disconnects from shared memory */
close(fd);                                 /* closes file descriptor */

exit(0);
```

Example – remove a shared memory area

■ A process removes the shared memory object

- After a successful `shm_unlink()`, attempts to `shm_open()` an object with the same name fail
- (unless `O_CREAT` was specified, in which case a new, distinct object is created)

```
int fd;
void *addr;                                /* pointer to shared memory */

fd = shm_open("/shmtest", ...);
ftruncate (fd, 100);
addr = mmap(NULL, 100, ...);

...                                        /* use – read/write */

munmap(addr, 100);                        /* disconnects from shared memory */
close(fd);                                /* closes file descriptor */
shm_unlink("/shmtest");                   /* removes from system */

exit(0);
```

Proper cleanup of POSIX shared memory

■ Why cleanup matters

- Prevents resource leaks and orphaned shared memory objects
- Ensures memory is fully released when no longer needed
- Avoids stale objects that may affect future executions
- Reduces the risk of resource exhaustion (e.g., accumulation in `/dev/shm/`)

■ Recommended cleanup order (good practice):

1. `munmap()` — detach the region from the process address space
2. `close()` — release the file descriptor
3. `shm_unlink()` — remove the shared memory name (performed by one designated process)

Proper cleanup of POSIX shared memory

■ Important notes

- `shm_unlink()` removes the name immediately, but the object persists until all processes have unmapped and closed it
- If a process terminates unexpectedly, the shared memory object may remain in the system until explicitly unlinked

The `fstat` function

```
#include <sys/stat.h>

int fstat(int fd, struct stat *buf);
```

- **Retrieves metadata about a shared memory object using its file descriptor**
- **Common uses:**
 - Determining the size of a shared memory object (especially if created by another process)
 - Verifying permissions before mapping
 - Monitoring changes in shared memory attributes

The `fstat` function

```
#include <sys/stat.h>

int fstat(int fd, struct stat *buf);
```

- `fd` is the file descriptor returned from `shm_open()`
- `buf` is the address of a `stat` structure where information will be placed
 - The structure will have info on the area, such as size, permissions, owner, etc.
- **Returns**
 - 0, if successful
 - -1, in case of error

The `stat` structure

- A system structure that acts as a metadata container
 - Holding essential file attributes such as size, permissions, and ownership

```
struct stat {  
    dev_t      st_dev;      /* ID of device containing file */  
    ino_t      st_ino;      /* inode number */  
    mode_t     st_mode;     /* protection */  
    nlink_t    st_nlink;    /* number of hard links */  
    uid_t      st_uid;      /* user ID of owner */  
    gid_t      st_gid;      /* group ID of owner */  
    dev_t      st_rdev;     /* device ID (if special file) */  
    off_t      st_size;     /* total size, in bytes */  
    blksize_t  st_blksize;  /* blocksize for file system I/O */  
    blkcnt_t   st_blocks;   /* number of 512B blocks allocated */  
    time_t     st_atime;    /* time of last access */  
    time_t     st_mtime;    /* time of last modification */  
    time_t     st_ctime;    /* time of last status change */  
};
```

Example – obtain information

```
int main(){
    int fd, ret;
    void *addr;
    struct stat shm_stat;

    fd = shm_open("/shmtest", O_CREAT|O_EXCL|O_RDWR, 0600);
    if(fd < 0) exit(1);

    ftruncate (fd, 100);
    addr = mmap(NULL,100,PROT_READ|PROT_WRITE,MAP_SHARED,fd,0);
    if(addr == MAP_FAILED) exit(1);

    if(fstat(fd, &shm_stat) < 0) exit (1);

    printf("mode = %d\n", shm_stat.st_mode);
    printf("size = %ld\n", shm_stat.st_size);
    ...
}
```

- Refer to `using_fstat.c` on Moodle for common uses of `fstat`

Exercise 1

■ Create two unrelated processes:

- Writer asks user the name of the place and a set of 10 temperature readings of the last 24 hours, putting all this information in a shared memory area
- Reader calculates and prints average temperature

■ The two processes may be executed simultaneously

- Synchronization is needed! Use busy-waiting flags *

■ Reader removes the shared memory area from the system

* This is intentionally inefficient and unsafe in real systems; It is used here for educational purposes before introducing semaphores

Exercise 2

- **Implement a program that creates a shared memory area for two arrays: $v[1000]$ and $max[10]$**
 - Initialize $v[]$ with random numbers
- **Create 10 children**
 - Each child should search the greatest value in 1/10 of $v[]$ and put it in $max[i]$ (where i is the child's index)
- **Parent waits for all children to terminate and calculates the maximum in $max[]$**
- **All processes should disconnect and close**
 - Parent should remove the shared memory area before terminating